Attention and transformers

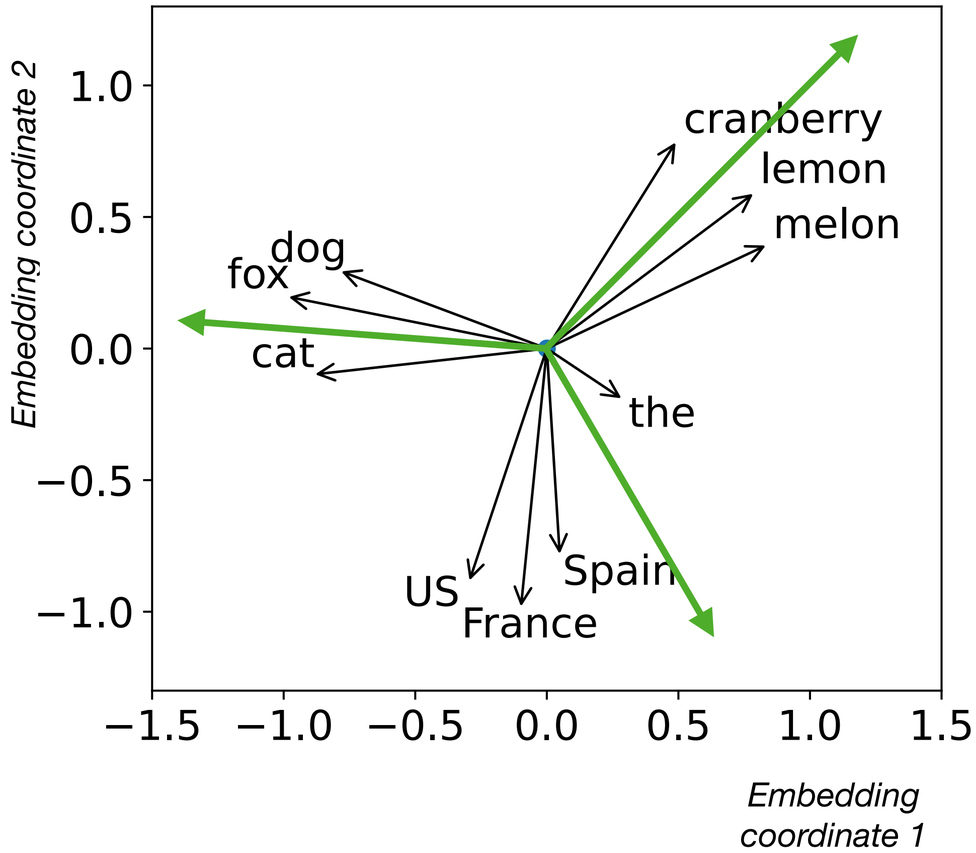
10-701 Introduction to Machine Learning

Geoff Gordon

Beyond word2vec

In the figure,, each of the three green queries retrieves primarily tokens for one group of concepts — e.g., {fox, dog, cat} on the left

- Embedding and attention are useful primitives, used in many more models beyond word2vec
- Interpret some hidden activation vectors as a database (like word2vec vocabulary)
  - $v_1, v_2, \ldots, v_K \in \mathbb{R}^d$



- Interpret other hidden activation vectors as a sequence of queries (like word2vec sliding window on input)
  - $q_1, q_2, \ldots, q_T \in \mathbb{R}^d$
- Can use networks with attention for all the sequence tasks mentioned above (many:many, many:1, …)

Attention (initial version)

- Look up queries in DB: for each query q_i
  - find attention weights over DB entries:

$$p_{ij} = \frac{\exp(\frac{1}{\sqrt{d}} q_i \cdot v_j)}{\sum_{j'} \exp(\frac{1}{\sqrt{d}} q_i \cdot v_{j'})}$$

  - return weighted average of DB entries:

$$\text{output}_i = \sum_j p_{ij} v_j$$

- Note scaling factor $1/\sqrt{d}$
  - doesn't change expressive power: can compensate by scaling q_i or v_j
  - but makes initialization less sensitive to query dimension

Attention: queries, keys, and values

$q_i, k_j \in \mathbb{R}^d \quad v_j \in \mathbb{R}^k$

- Generalization: DB contains *key-value pairs* (k_j, v_j)
  - compare query to k_j but return v_j in case of match
- Compare query q_i to key k_j:

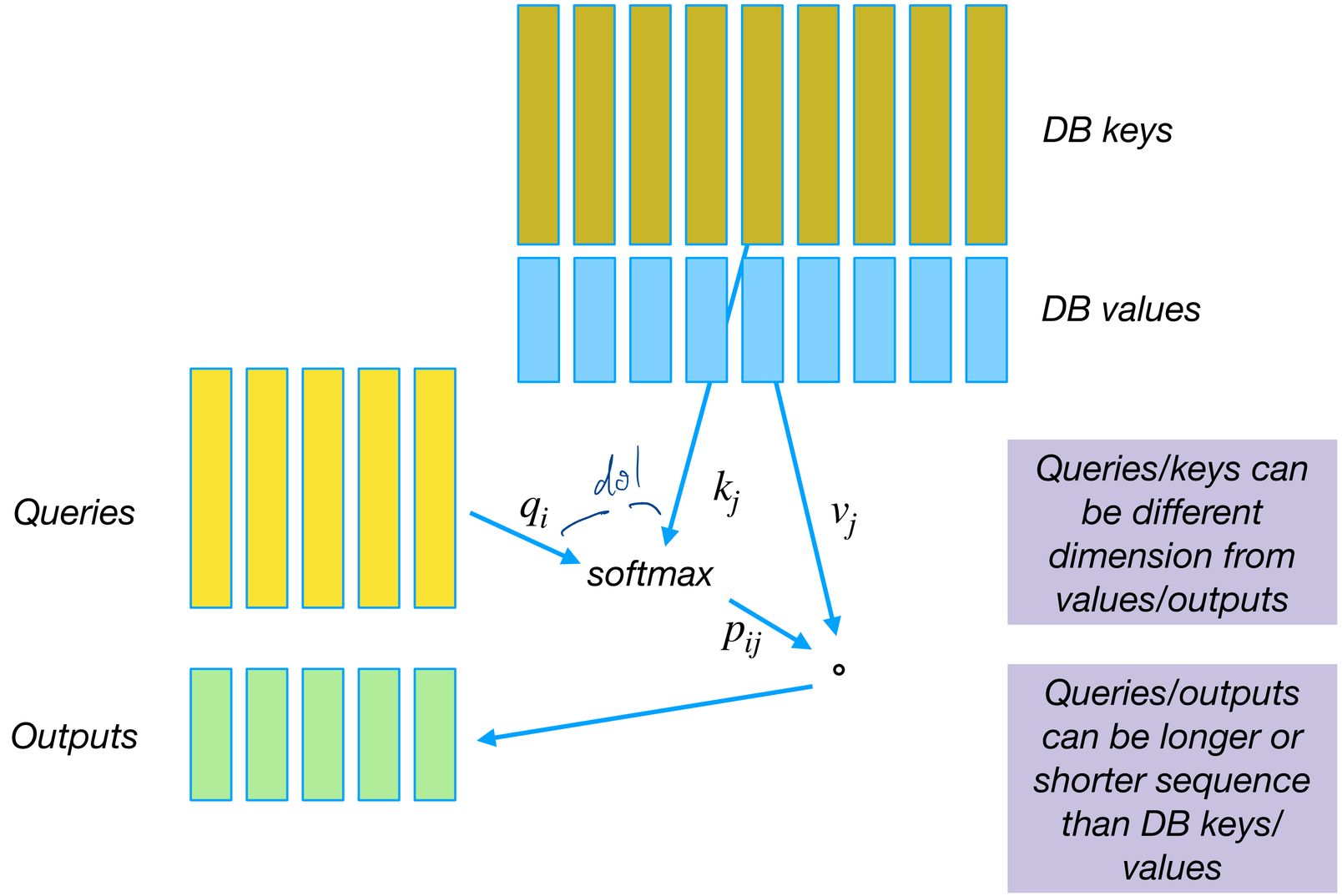
$$\text{attention weights } p_{ij} = \frac{\exp(\frac{1}{\sqrt{d}} q_i \cdot k_j)}{\sum_{j'} \exp(\frac{1}{\sqrt{d}} q_i \cdot k_{j'})}$$

- Return weighted average of values v_j:

$$\text{output}_i = \sum_j p_{ij} v_j \in \mathbb{R}^k$$

- Decouple what we look up from what we return
  - v_j can even be a different dimension

Attention



- Called an *attention head:* maps sequence of queries to sequence of outputs, using DB (key-value pairs) as reference

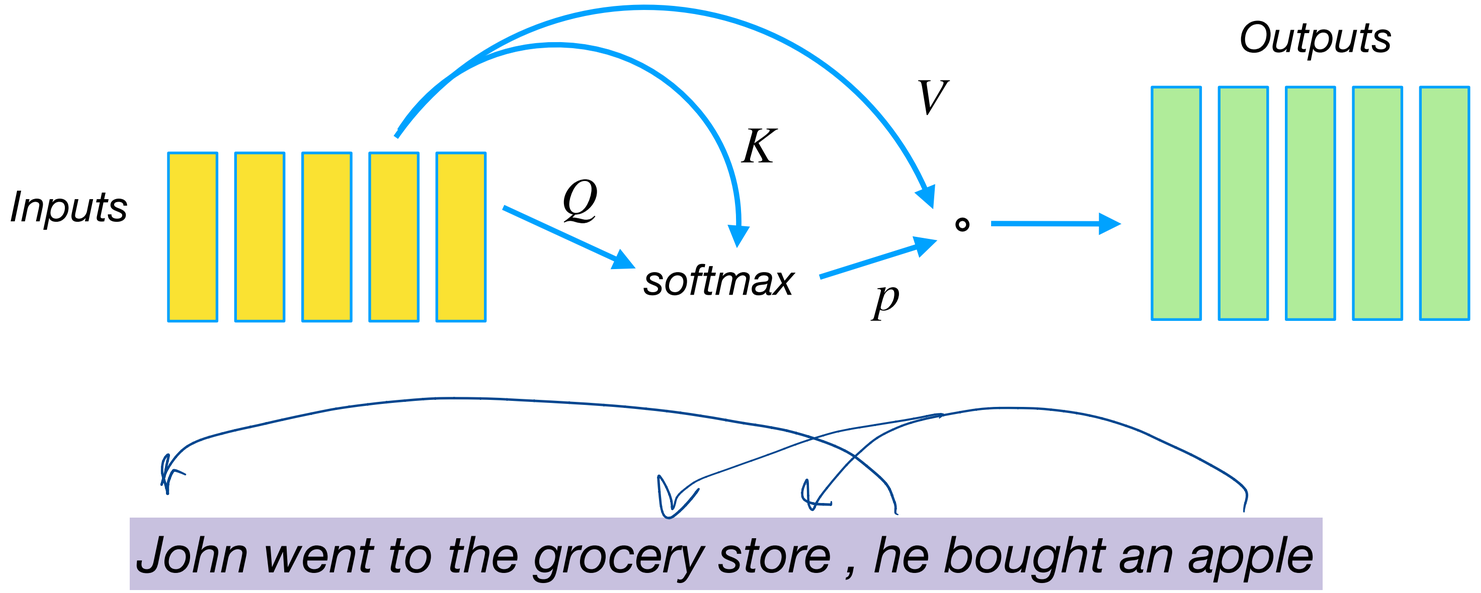
Self-attention

- Single sequence $x_i \in \mathbb{R}^d$ (e.g., input tokens, or output of previous layer) acts as *both* DB and queries
- Transform with learnable parameter matrices Q, K, V
  - queries $q_i = Qx_i$, keys $k_i = Kx_i$, values $v_i = Vx_i$
- Once we get DB and queries from x_i, proceed as before

$$p_{ij} = \frac{\exp(Qx_i \cdot Kx_j)}{\sum_{j'} \exp(Qx_i \cdot Kx_{j'})} \qquad \text{output}_i = \sum_j p_{ij} V x_j$$

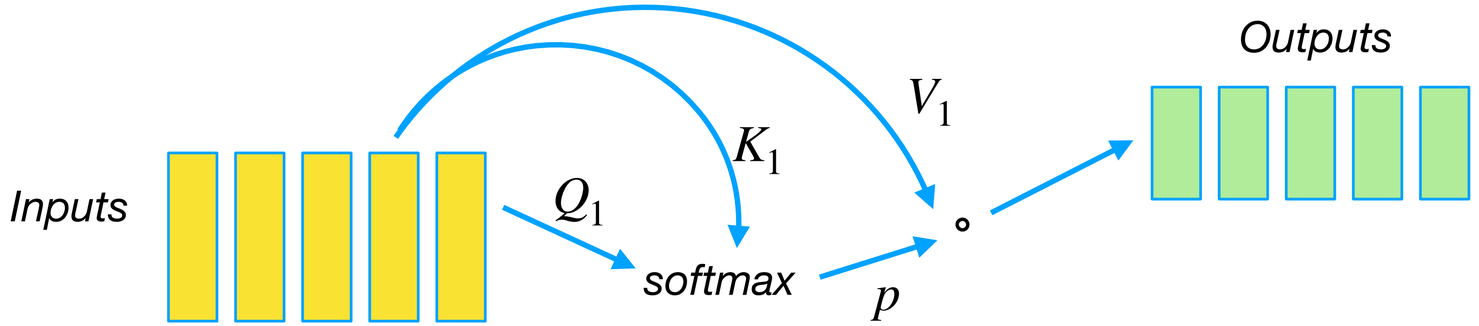
- Queries q_i and keys k_i have to be same dimension, but can differ from dimension of inputs x_i and outputs v_i
  - Q, K map x_i to a common dimension for dot product; V maps x_i to desired output dimension

Self-attention



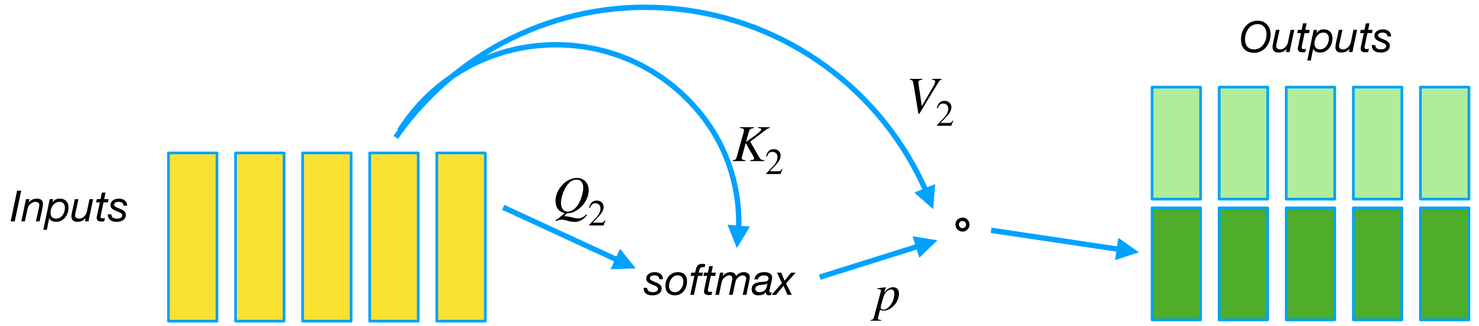
- Database and query sequence come from same source:
  - for each token in a document, find other relevant tokens
  - e.g., referent of a pronoun [he → John]
  - e.g., similar topic [apple → grocery, store]
- Extremely parallelizable! :) But $O(T^2)$:(

Multi-head attention



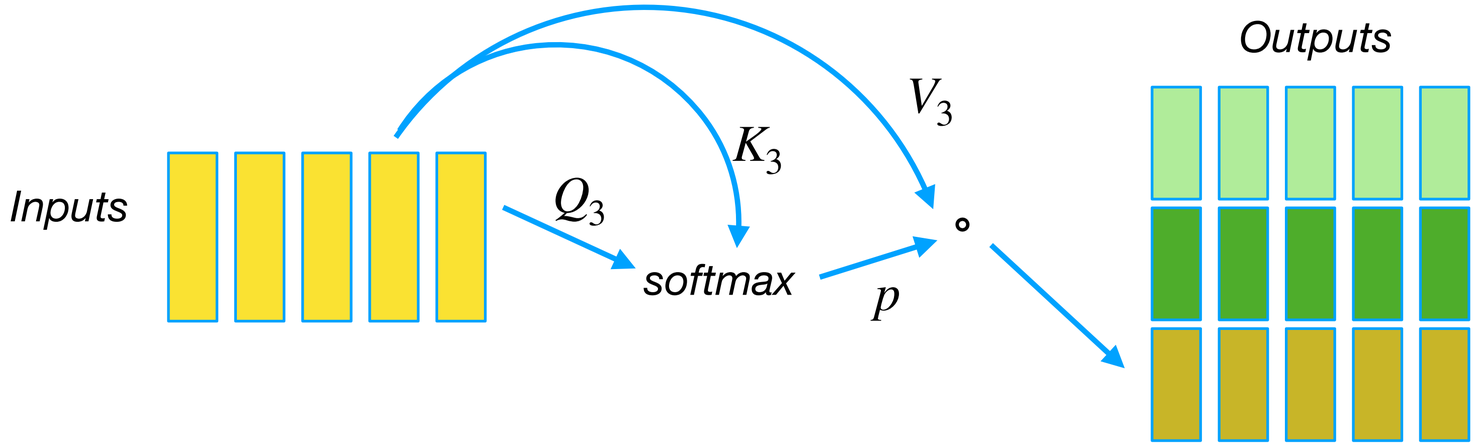
- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

Multi-head attention



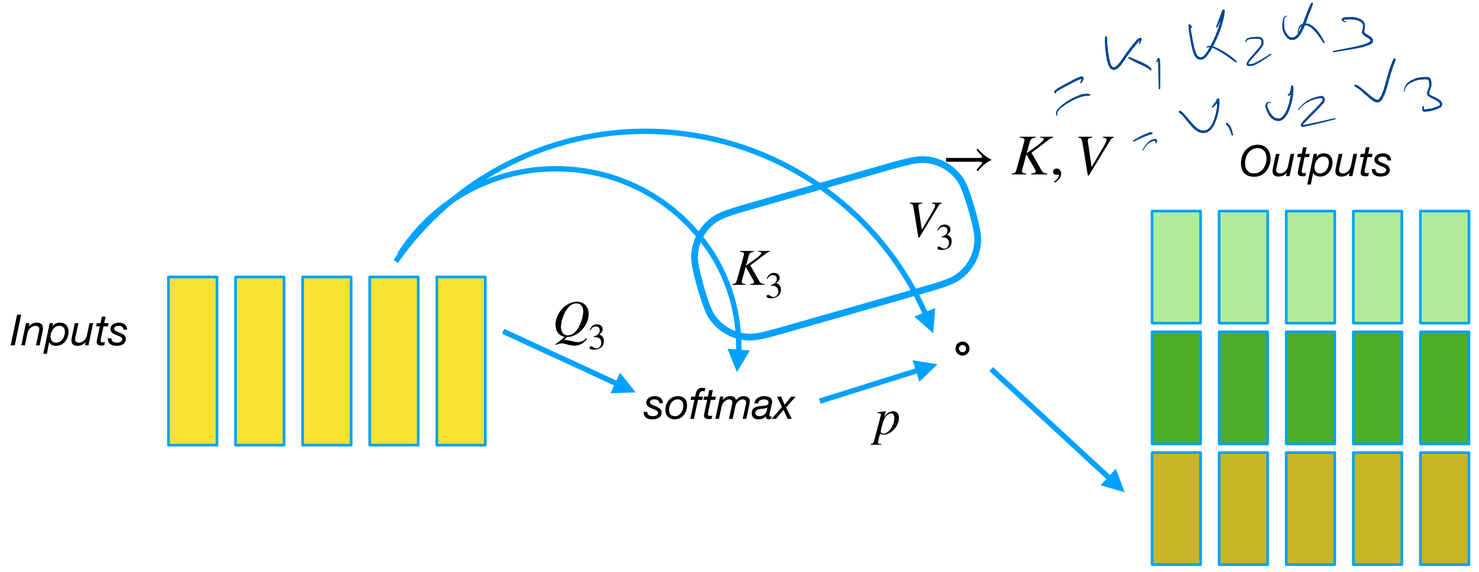
- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

Multi-head attention



- Do attention several times in parallel, concatenate results
  - often make each head's output value shorter so total dimension remains the same

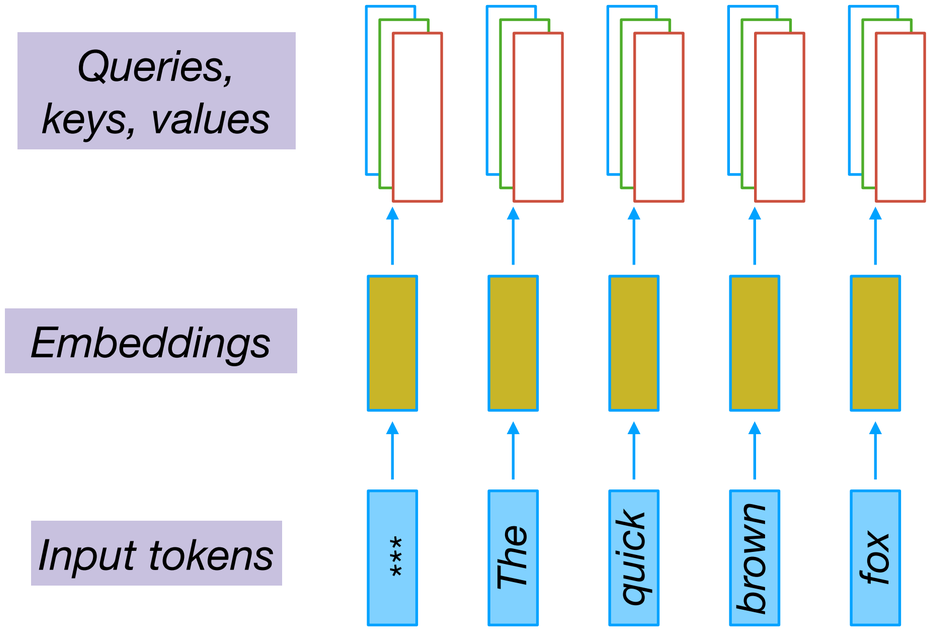
GQA



- Can share parameters: e.g., keep K, V constant in blocks while Q varies (***grouped query attention***, GQA)
  - saves memory, computation, and model size

Positional encoding

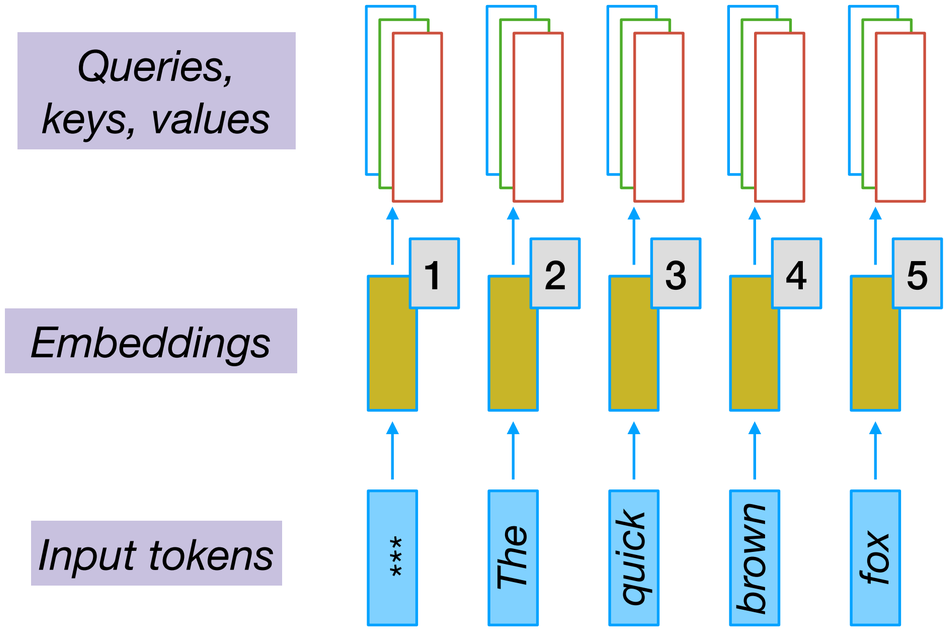
- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

Positional encoding

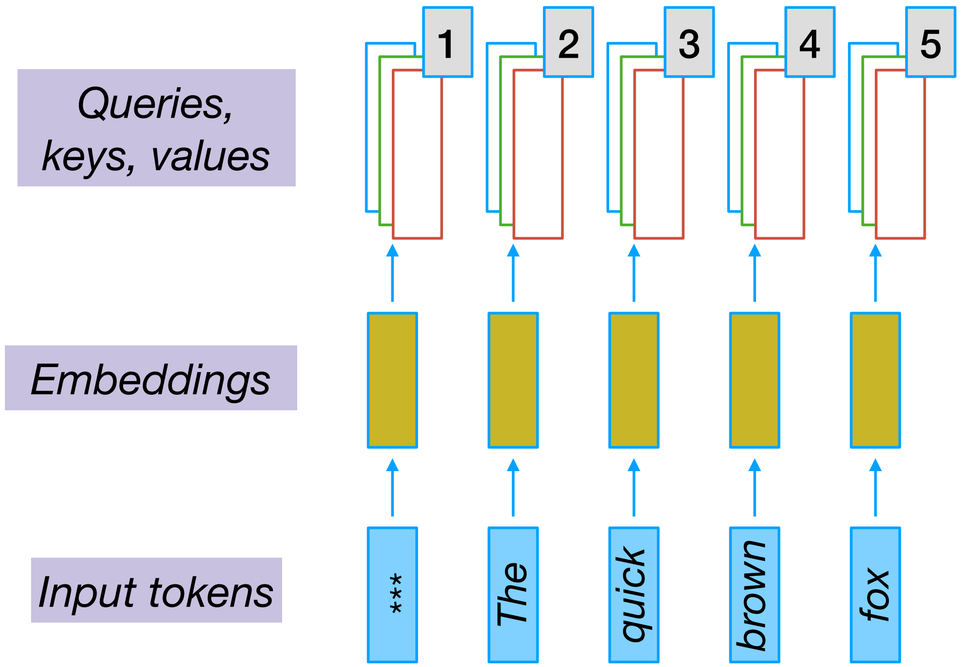
- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

Positional encoding

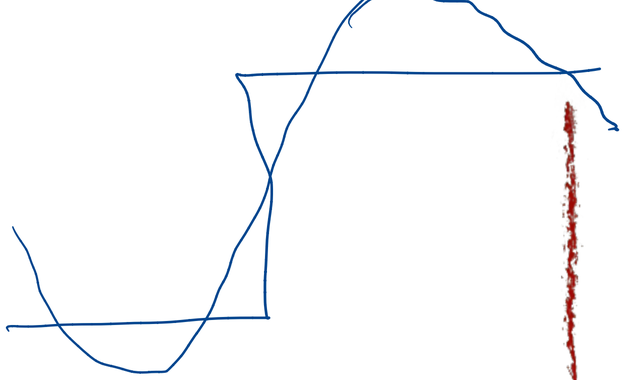
- Attention is permutation-invariant for DB, equivariant for queries
  - order of DB doesn't affect prediction
  - reordering queries reorders predictions the same way



- Often want only sort-of-invariance: *generalize* from position to position, but don't *force* identical behavior
- Solution: encode token position somehow in embedding or attention layer
  - network can pay attention or ignore, whatever works best

What makes a good positional encoding?

- Consider a query q_i at position i and key k_j at position j
  - imagine varying j: keep the same query and key, but the key shows up at different positions
  - a positional encoding at attention layer alters $q_i \to q_i^{\text{enc}}$ and $k_j \to k_j^{\text{enc}}$
  - which lets the dot product $q_i^{\text{enc}} \cdot k_j^{\text{enc}}$ vary as we shift j
  - therefore attention weight p_{ij} also varies
- An ideal positional encoding would let us represent any translation-invariant score function $q_i^{\text{enc}} \cdot k_j^{\text{enc}} = f(j - i)$
  - attend iff k shows up at previous token, or attend iff k shows up at position +10 to +15, or …



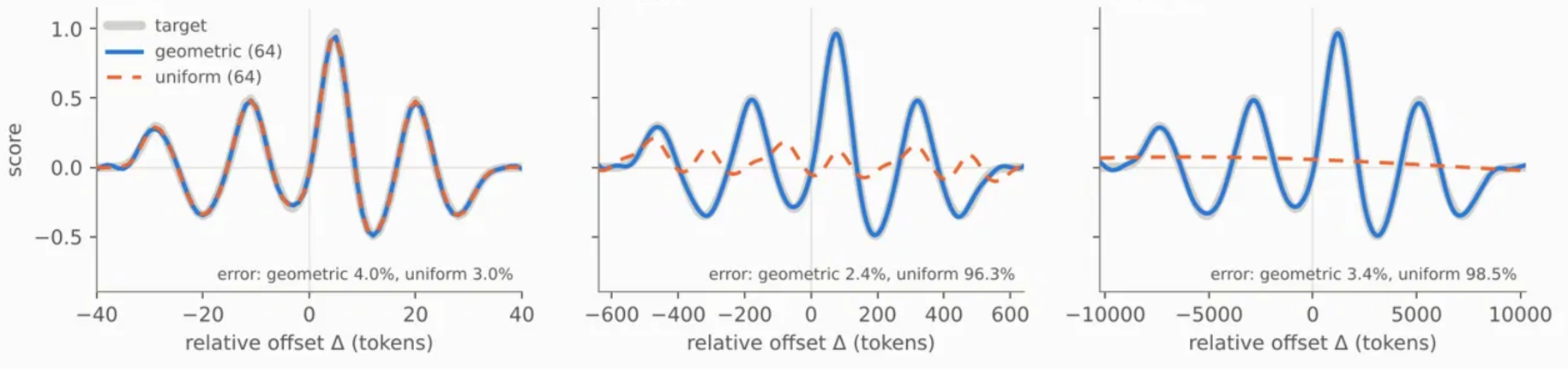
Representing functions: Fourier basis

- One good way to represent functions is a Fourier basis:

$$f(\Delta) \approx \sum_{b=1}^{d} \alpha_b \sin(\theta_b \Delta) + \beta_b \cos(\theta_b \Delta) \qquad \Delta = j - i$$

  - with d different (fixed) frequencies θ_b, weights α_b, β_b
- Which frequencies?
  - different choices lead to different representational bias
  - Fourier transform uses uniform sampling of frequencies: $\frac{2\pi}{N}, \frac{2(2\pi)}{N}, \frac{3(2\pi)}{N}, \ldots$ for a function with inputs $0 \ldots N-1$
  - could also do geometric (uniform on a log scale): $B^{\frac{1}{N}}, B^{\frac{2}{N}}, B^{\frac{3}{N}}, \ldots$ for some base $B < 1$
  - many other possible choices

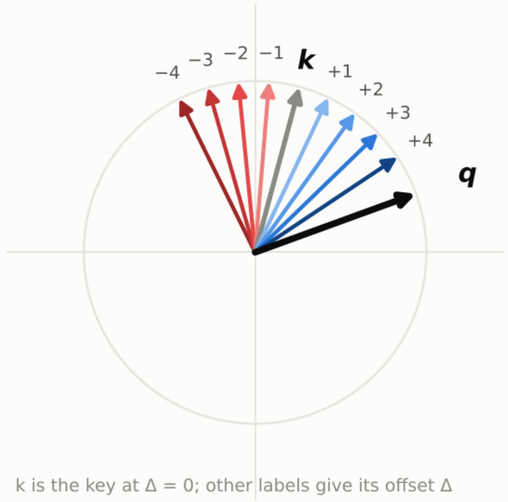
Effect of frequency sampling

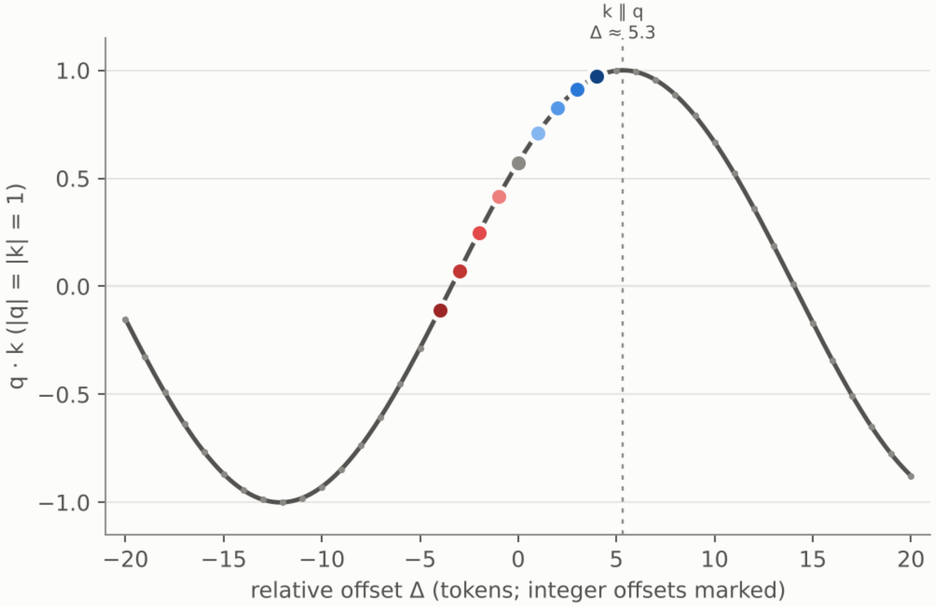


- We'll use geometric: gives *scale-invariant* accuracy
  - error for representing a function $f(\Delta)$ is about the same as for $f(\alpha\Delta)$ for any scale $\alpha > 0$ in a wide range
- Plot: given a target $f(\Delta)$ that is nonzero in –40..40,
  - show best fit for f in Fourier basis with geometric vs. uniform sampling
  - compare after scaling f to cover –600..600 or –10k..10k

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





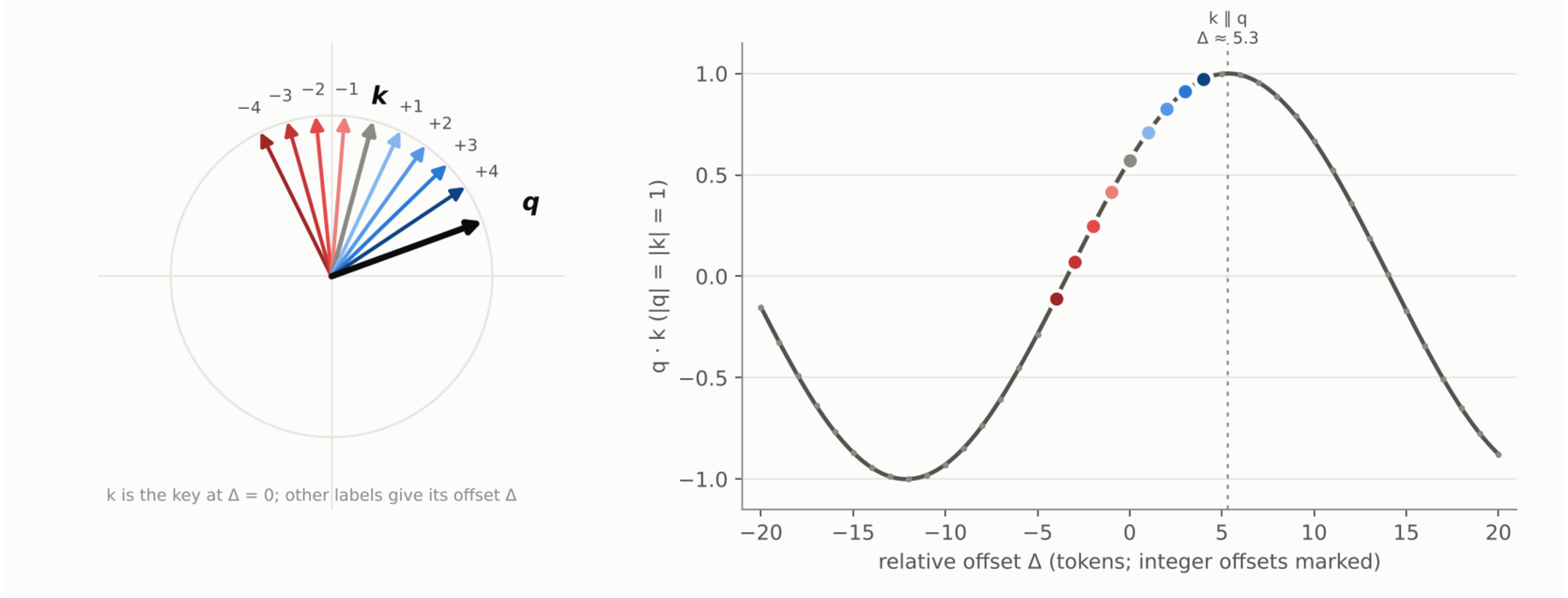
- Suppose $q, k \in \mathbb{R}^2$, with q at token position i and k at j
- Pick a fixed angle θ
- Transform by rotating in increments of θ
  - $q^{\text{enc}} = R_{i\theta} q$ and $k^{\text{enc}} = R_{j\theta} k$
  - plot: fix i, sweep j

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?



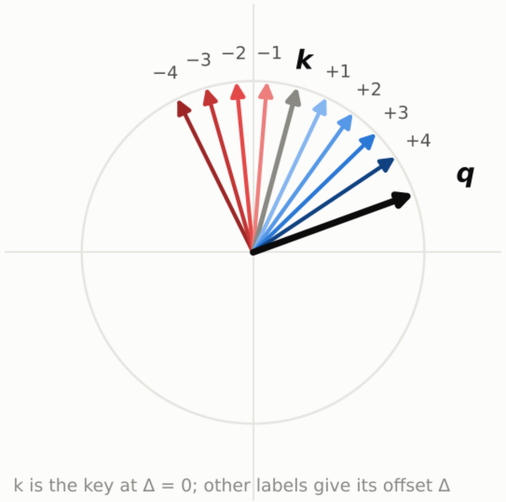
- Only the difference $\Delta = j - i$ matters:
  - rotate clockwise with $R_\theta^{-1} = R_{-\theta} = R_\theta^\top$
  - so $(R_{j\theta}k) \cdot (R_{i\theta}q) = R_{(j-i)\theta}k \cdot q$

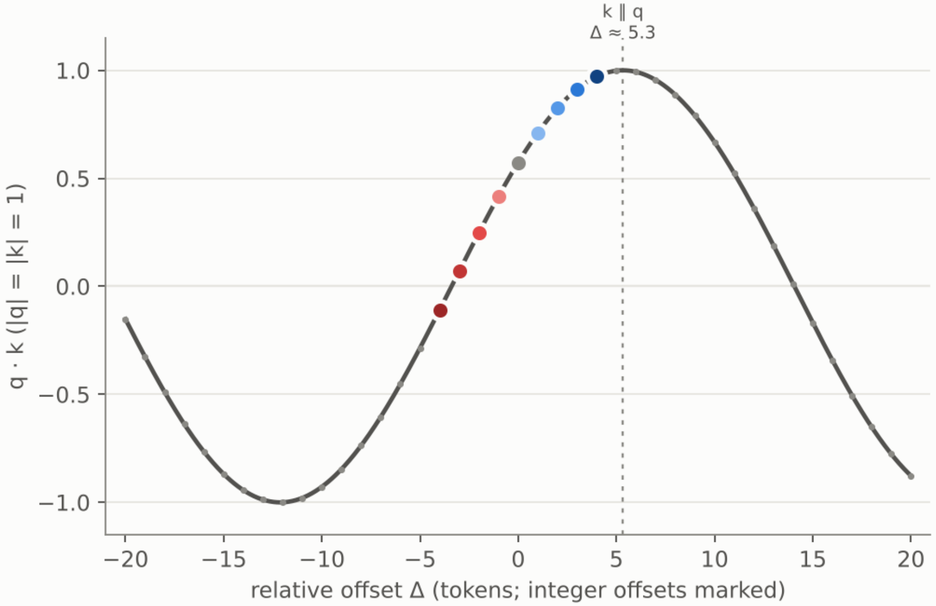
$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





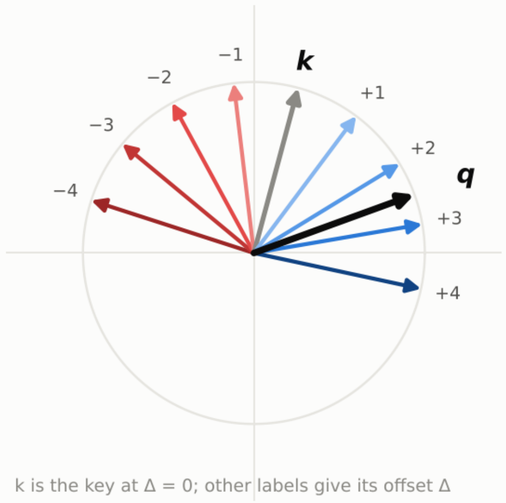
- Gives us a sinusoid:
  - frequency $= \theta$
  - amplitude (weight) $= \|q\| \, \|k\|$
  - phase = initial angle ϕ between q and k
- Called ***Rotary Positional Encoding***

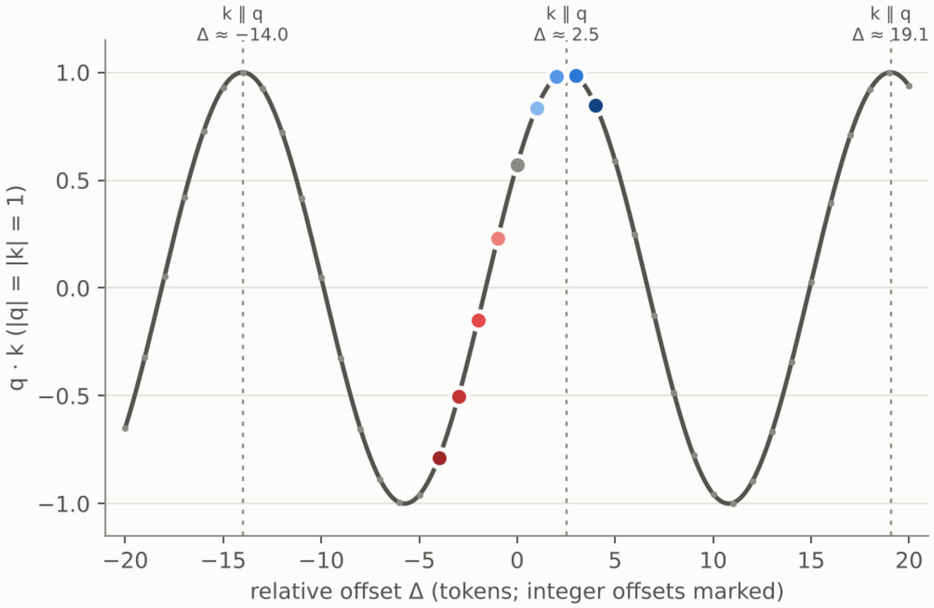
$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





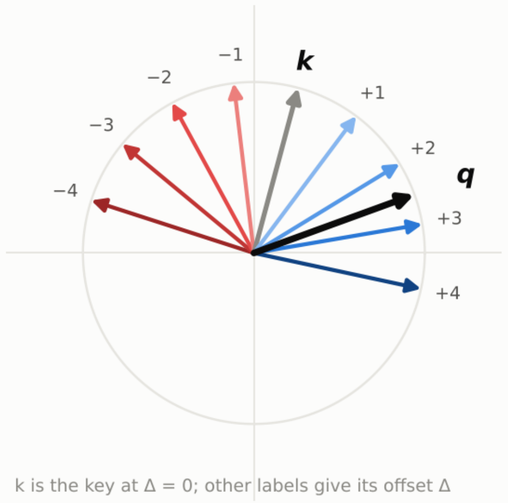
- Gives us a sinusoid:
  - frequency $= \theta$
  - amplitude (weight) $= \|q\| \, \|k\|$
  - phase = initial angle ϕ between q and k
- Called ***Rotary Positional Encoding***

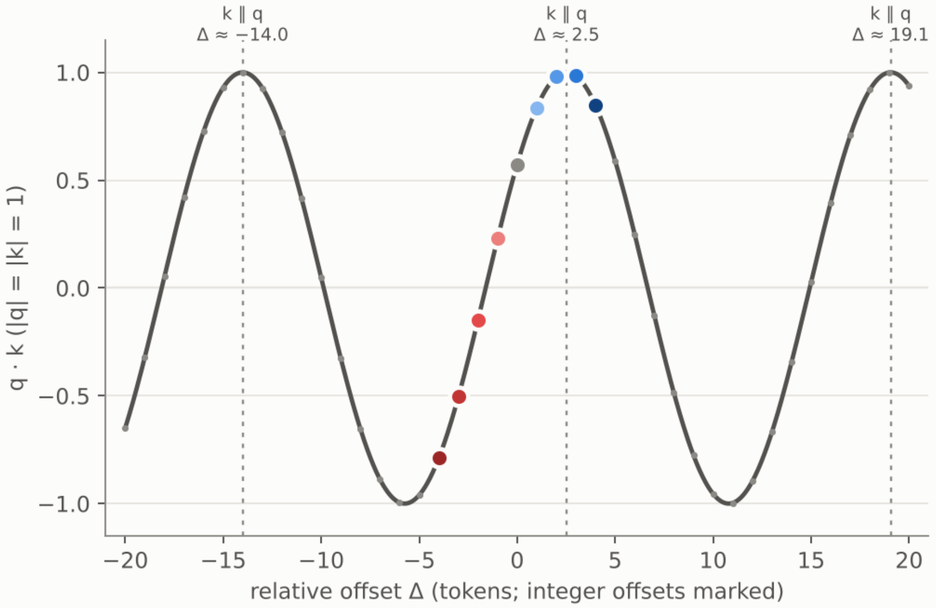
$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: one frequency

how do we get sin or cos out of dot product of embeddings?





- Note it's equivalent to have both sin and cos with weights α, β or to have just sin with weight γ, phase ϕ:

$$\gamma \sin(\theta\Delta + \phi) = \gamma \sin(\theta\Delta)\cos(\phi) + \gamma \cos(\theta\Delta)\sin(\phi)$$
$$= \alpha \sin(\theta\Delta) + \beta \cos(\theta\Delta)$$

  - by angle-sum formula
  - polar ↔ Cartesian

$$R_\theta = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$$

rotates CCW by θ

RoPE: multiple frequencies

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- Sample a geometric grid of frequencies $\theta_b = B^{\frac{b}{N}}$
- Divide q and k into blocks of 2 adjacent coordinates
- Encode by rotating block b by a multiple of θ_b
  - by same argument as above: attention score = sum of N sinusoids of different frequency, amplitude, and phase

RoPE: expressive power

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one $\rightarrow$ implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

RoPE: expressive power

variant: partial RoPE — only rotate some blocks of q and k; remaining blocks keep globally useful information

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one → implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

RoPE: expressive power

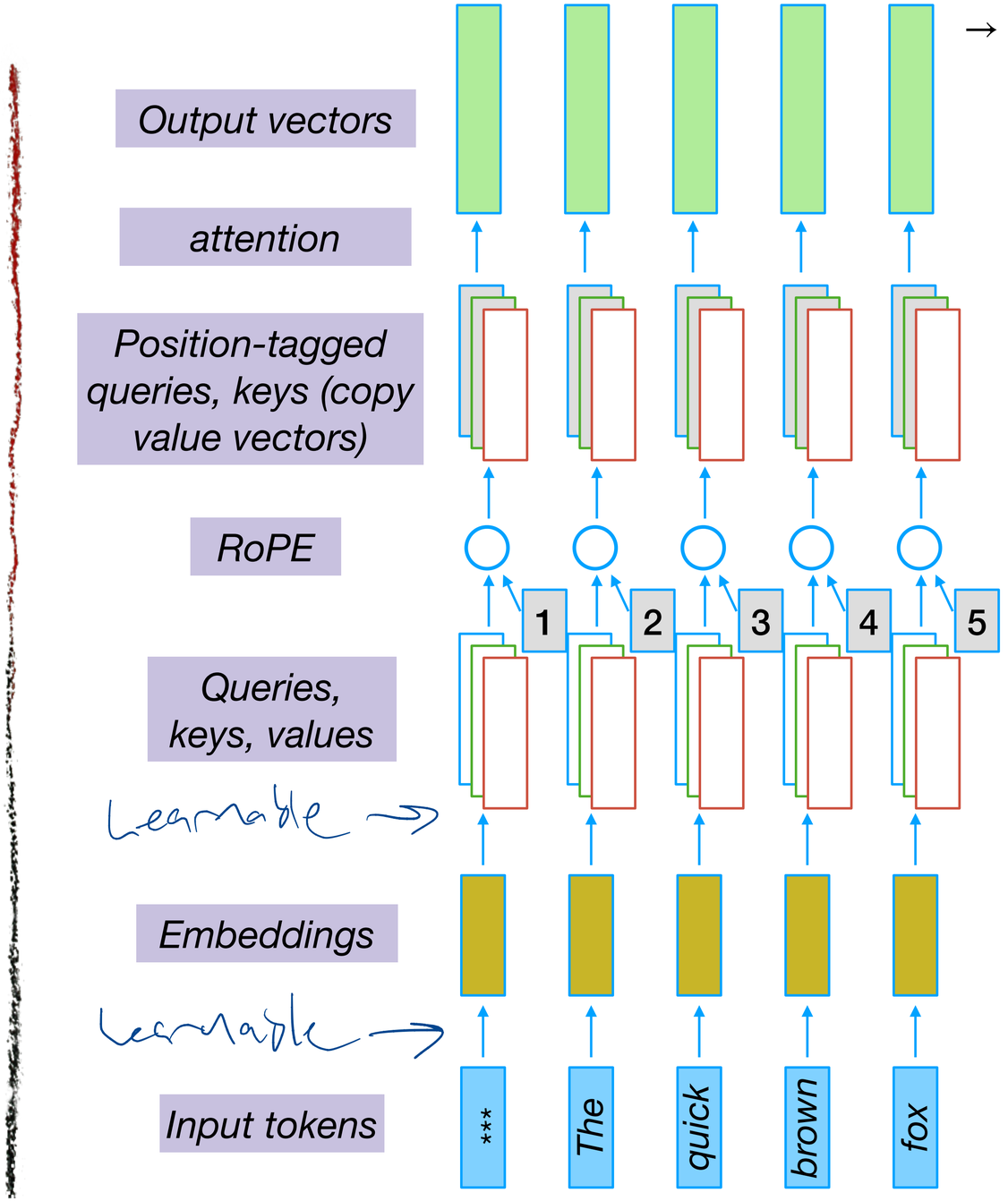
variant: partial RoPE — only rotate some blocks of q and k; remaining blocks keep globally useful information

If fraction goes to zero, it's called NoPE

$$q = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \\ q_4 \\ q_5 \\ q_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} q_{1:2} \\ \\ q_{3:4} \\ \\ q_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow q^{\text{enc}} = \begin{pmatrix} R_{i\theta_1} q_{1:2} \\ \\ R_{i\theta_2} q_{3:4} \\ \\ R_{i\theta_3} q_{5:6} \\ \\ \vdots \end{pmatrix} \qquad k = \begin{pmatrix} k_1 \\ k_2 \\ k_3 \\ k_4 \\ k_5 \\ k_6 \\ \vdots \end{pmatrix} = \begin{pmatrix} k_{1:2} \\ \\ k_{3:4} \\ \\ k_{5:6} \\ \\ \vdots \end{pmatrix} \rightarrow k^{\text{enc}} = \begin{pmatrix} R_{j\theta_1} k_{1:2} \\ \\ R_{j\theta_2} k_{3:4} \\ \\ R_{j\theta_3} k_{5:6} \\ \\ \vdots \end{pmatrix}$$

- N sinusoids of different frequency, amplitude, and phase
  - frequencies are fixed ahead of time: θ_b
  - for each q, can design a k that chooses amplitude and phase of each one → implements any attention score function $f(\Delta)$ to the limit of our basis (sin and cos at θ_b)
- Different q, k pairs have to share (increase error and/or use similar attention score functions)
  - also share with other functions of embedding

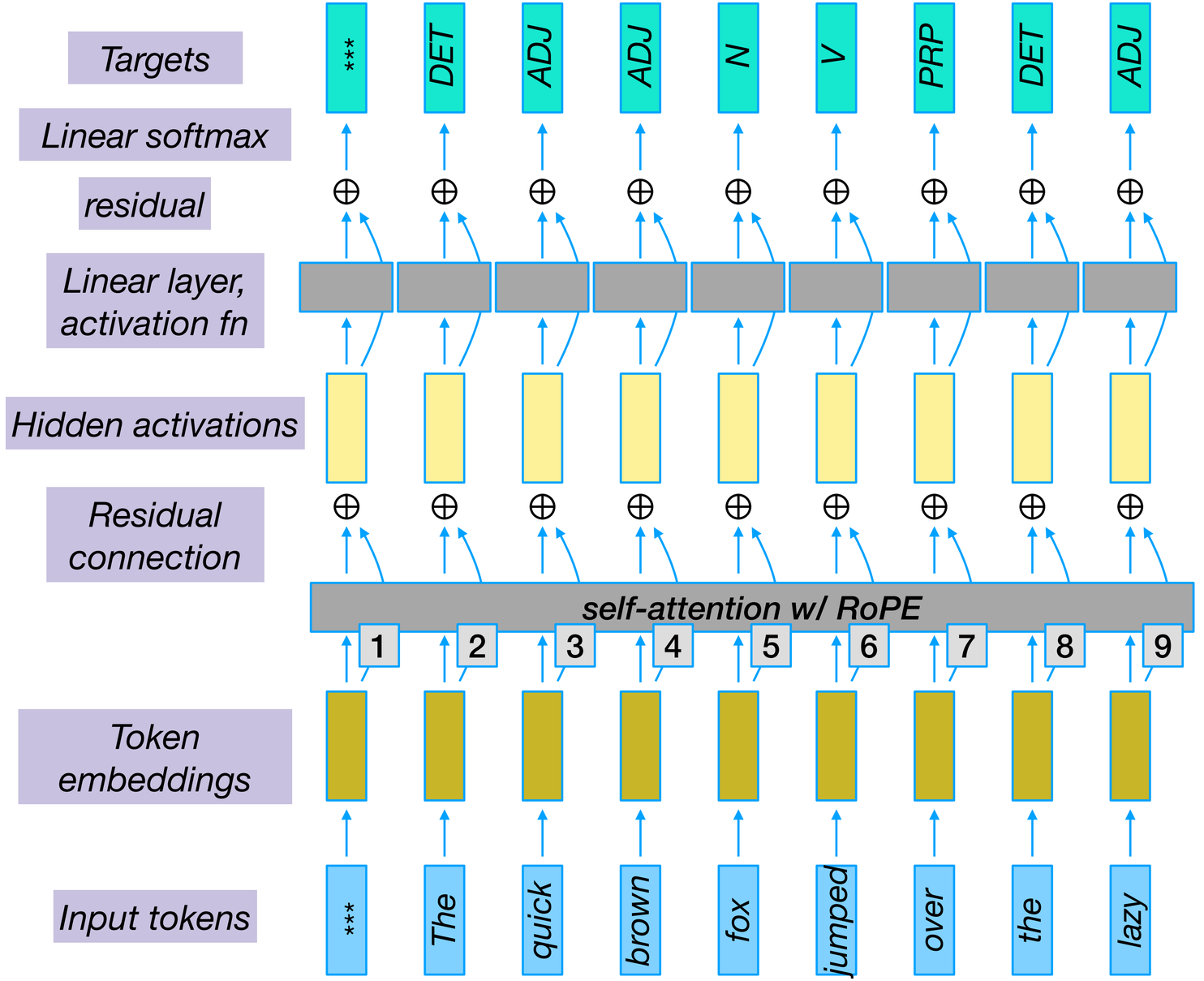
Attention with RoPE



- Put it together: look up embedding for each input token; compute queries, keys, and values w/ weight matrices K, Q, V; rotate blocks in each query/key to encode position (token index); dot product, softmax, and sum for attention

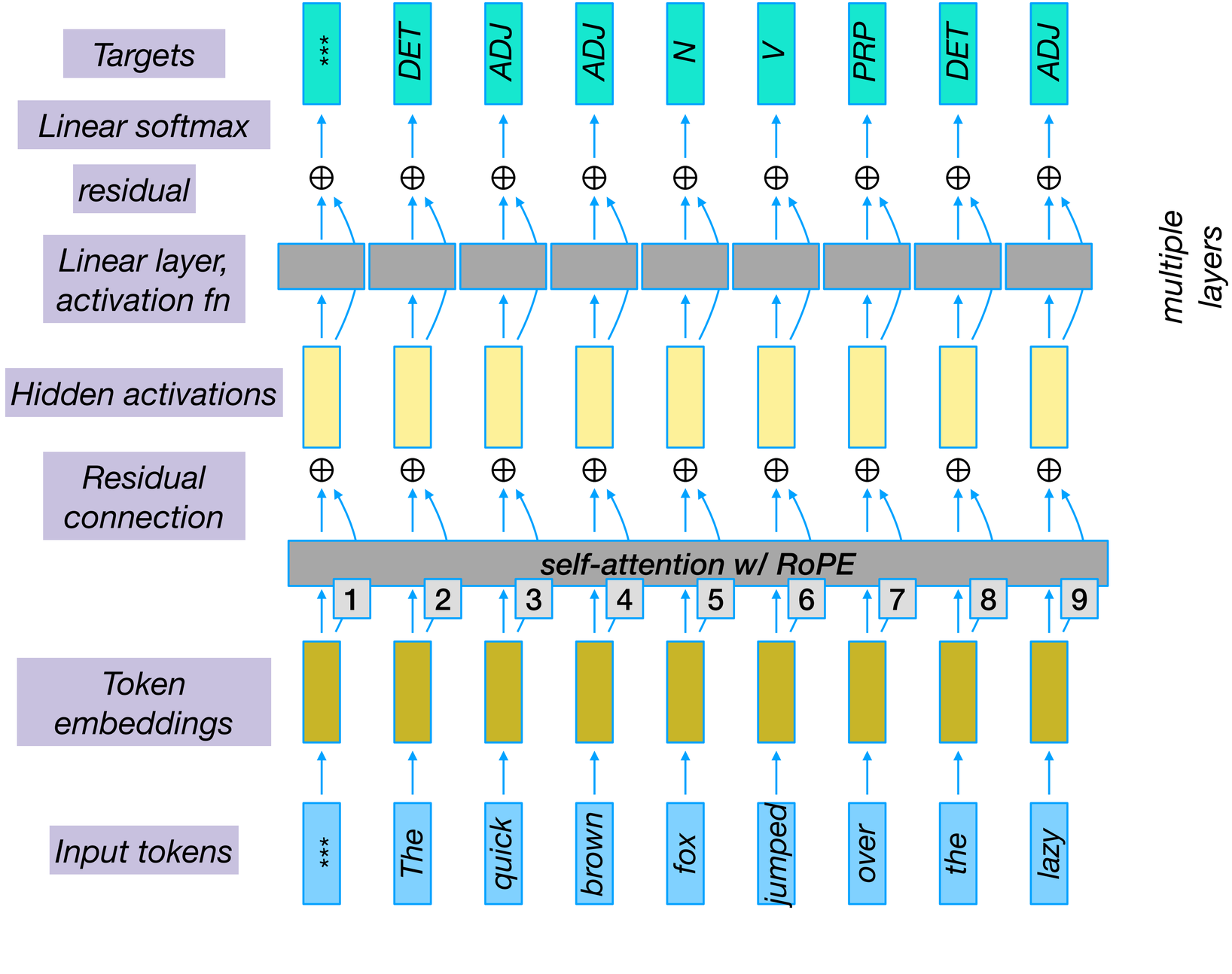
Predicting targets: pointwise MLP

Here, for POS tagging



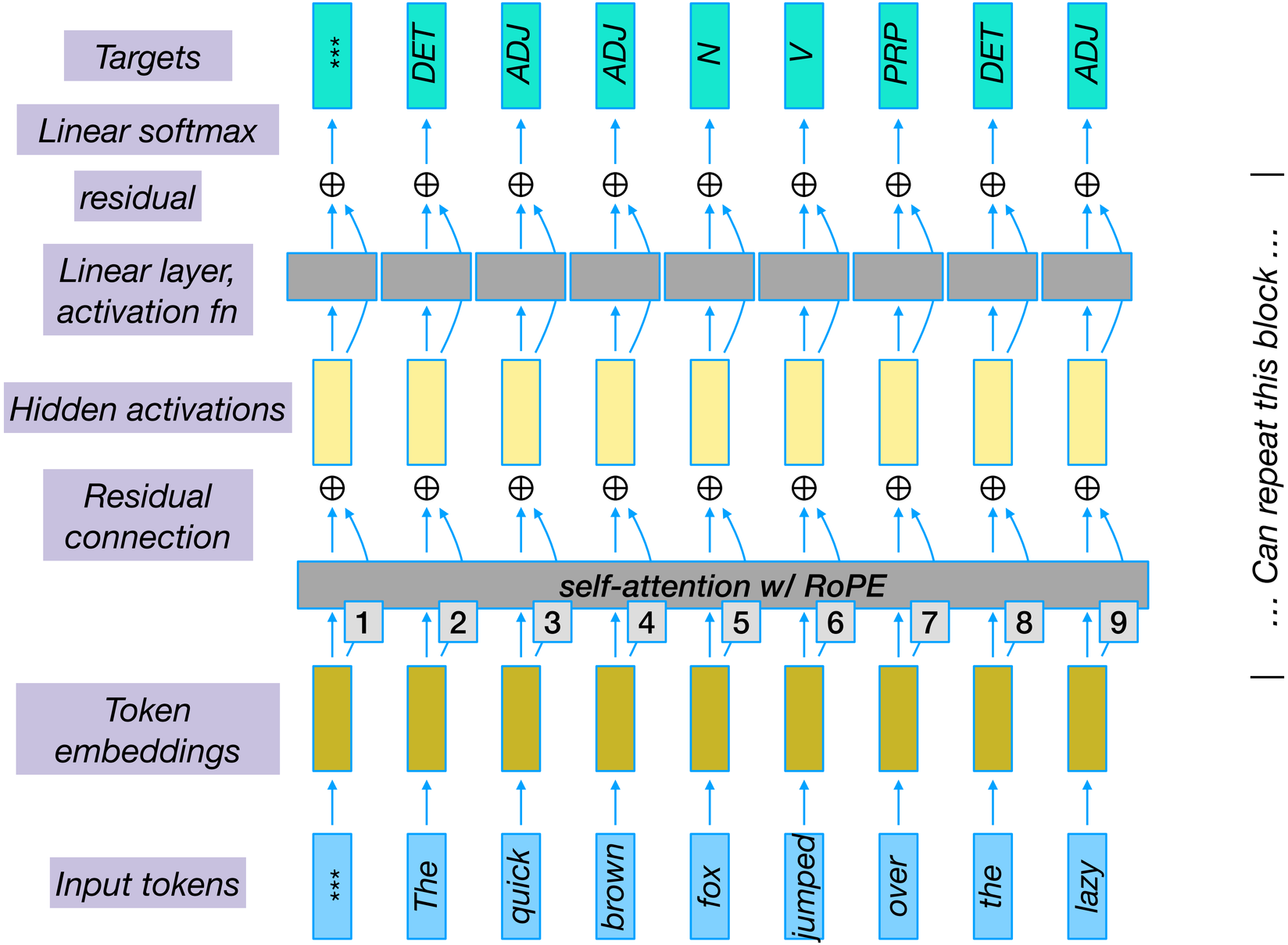
Predicting targets: pointwise MLP

Here, for POS tagging



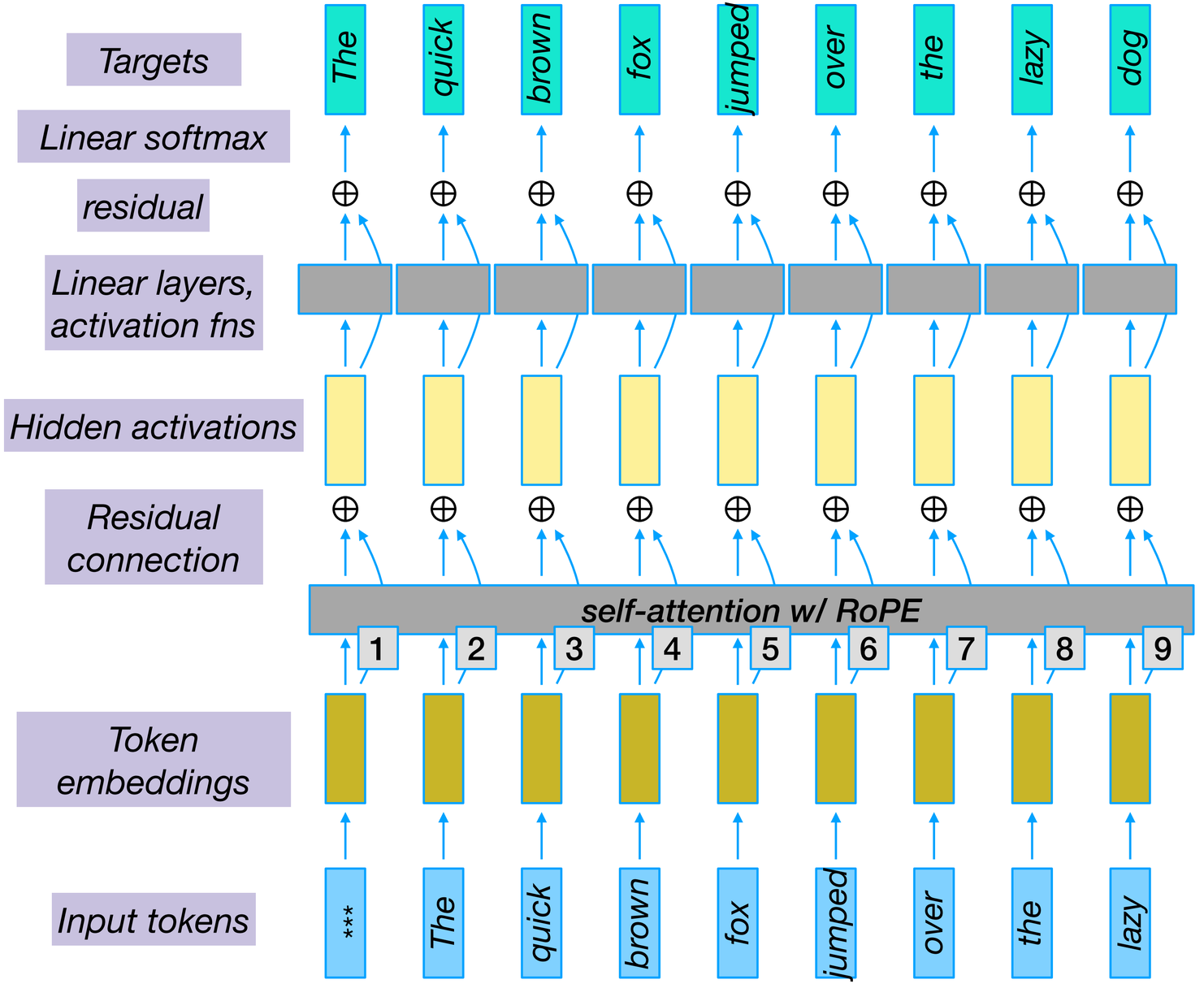
Predicting targets: pointwise MLP

Here, for POS tagging



What about next-token prediction?

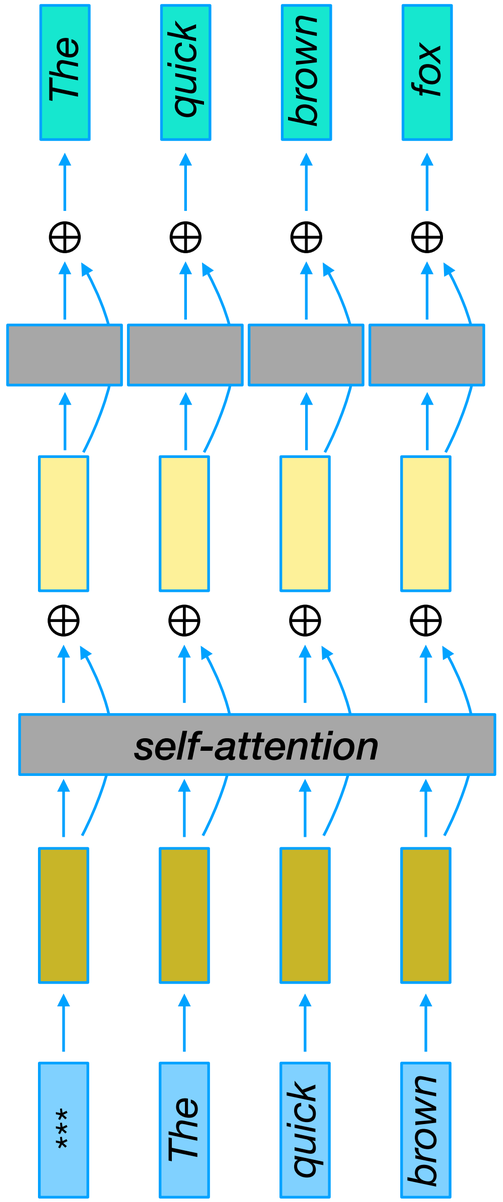
Can you spot the problem?



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

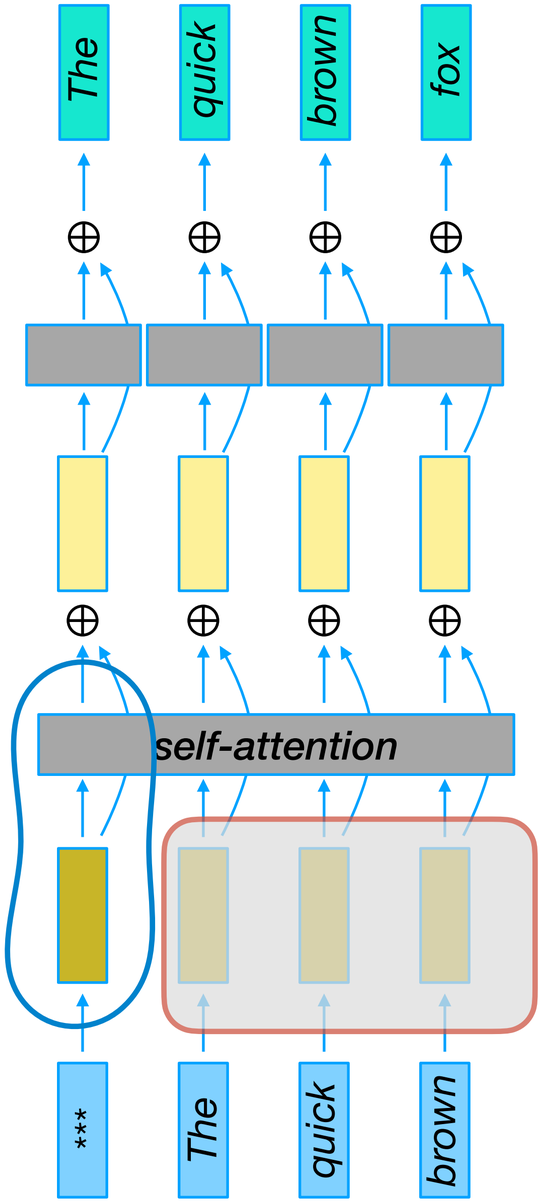
$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

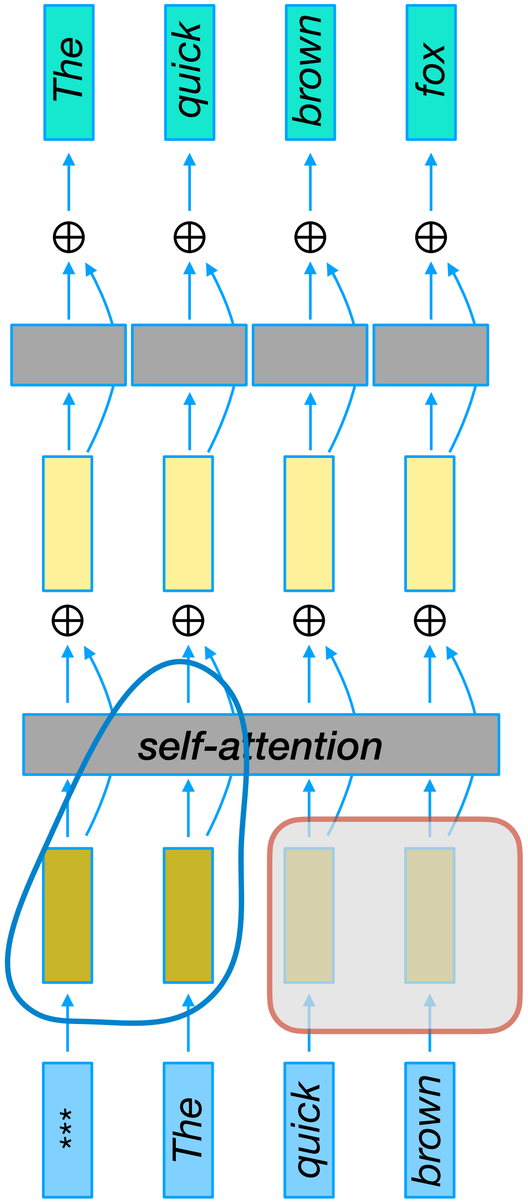
$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Q x_i \cdot K x_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$



Attention mask

- Only let attention look at a subset of positions (*masked* attention)
- Here, own position and earlier in the sentence: causally masked self-attention

$$p_{ij} = \begin{cases} \dfrac{\exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_j)}{\sum_{j'=1}^{i} \exp(\frac{1}{\sqrt{d}} Qx_i \cdot Kx_{j'})} & 1 \le j \le i \le T \\ 0 & \text{o/w} \end{cases}$$

